Operating Systems

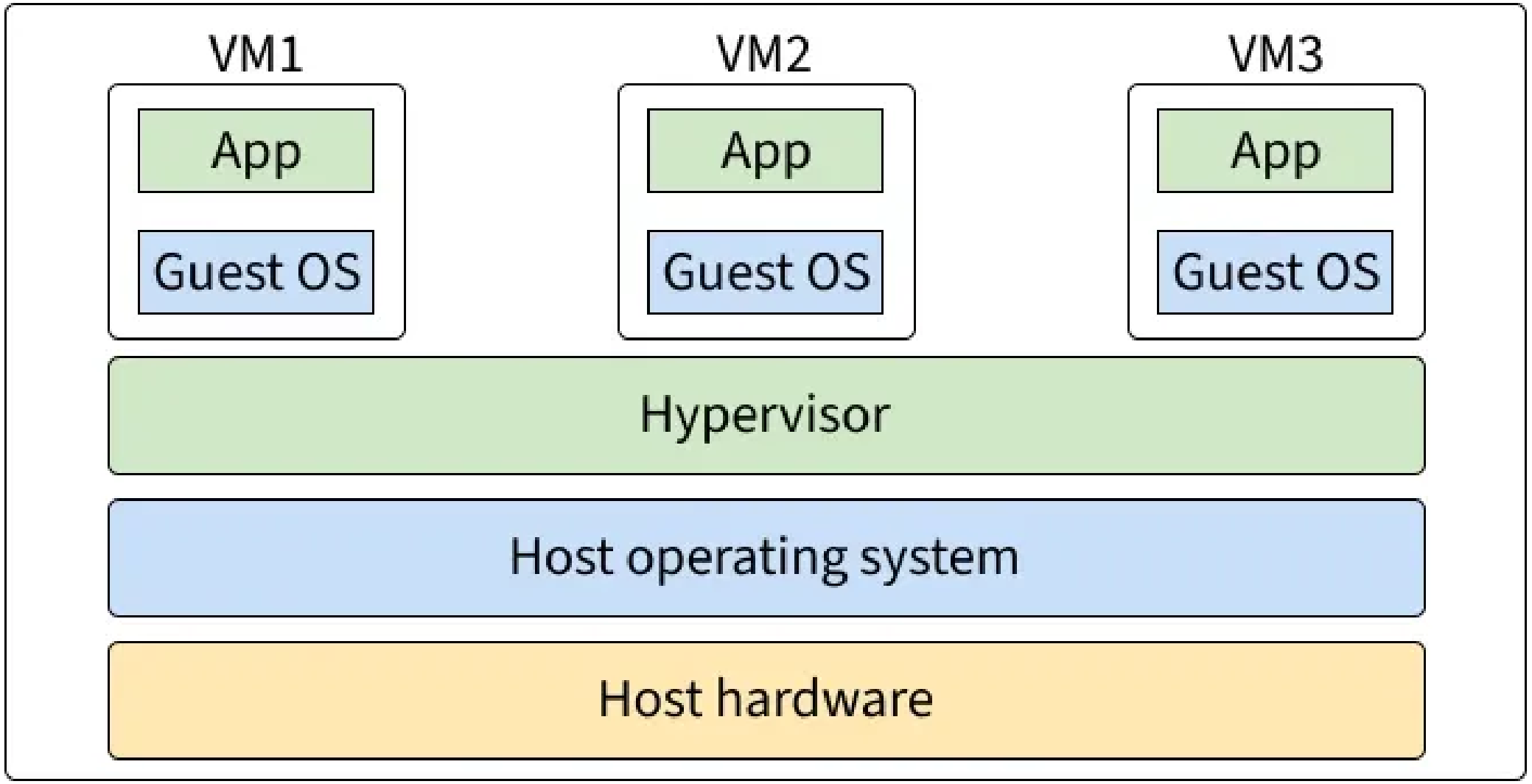
Dmitry Zaitsev

Lecture 14: Concepts of a virtual machine and container. Case study: Docker containers.

Virtual Machine (VM) – Software model of computer hardware

- Emulator of a given hardware (museum of running computers)
- Specify a VM as a definite set of resources – processor type, RAM size, HDD/SDD
- Provide mapping of VM resources into actual resources of your computer within host OS
- Install OS on the VM
- Install application software on the VM
- Work on the VM

Virtual machines



VM software (hypervisors)

- Oracle VirtualBox, https://www.virtualbox.org
- Microsoft Hyper-V, https://docs.microsoft.com/en-us/virtualization/hyper-v-on-windows/about
- Citrix XenServer, https://www.xenserver.com
- VMware Workstation Pro, https://www.vmware.com/products/desktop-hypervisor
- QEMU, https://www.qemu.org
- Xen Project, https://xenproject.org
- Others: WLS, Wine

Hypervisor and VM

- A hypervisor is the manager software that creates and runs multiple isolated virtual environments (VMs) on one physical computer, while a Virtual Machine (VM) is the guest software computer itself, complete with its own OS and apps, running on the hypervisor, sharing the host's hardware.
- Hypervisor: Software, firmware, or hardware that creates, runs, and manages VMs. Allocates physical resources (CPU, memory, storage) to VMs and keeps them isolated.
- Virtual Machine: A software-based, isolated emulation of a physical computer. Runs its own operating system (like Windows, Linux) and applications, acting like a separate physical machine

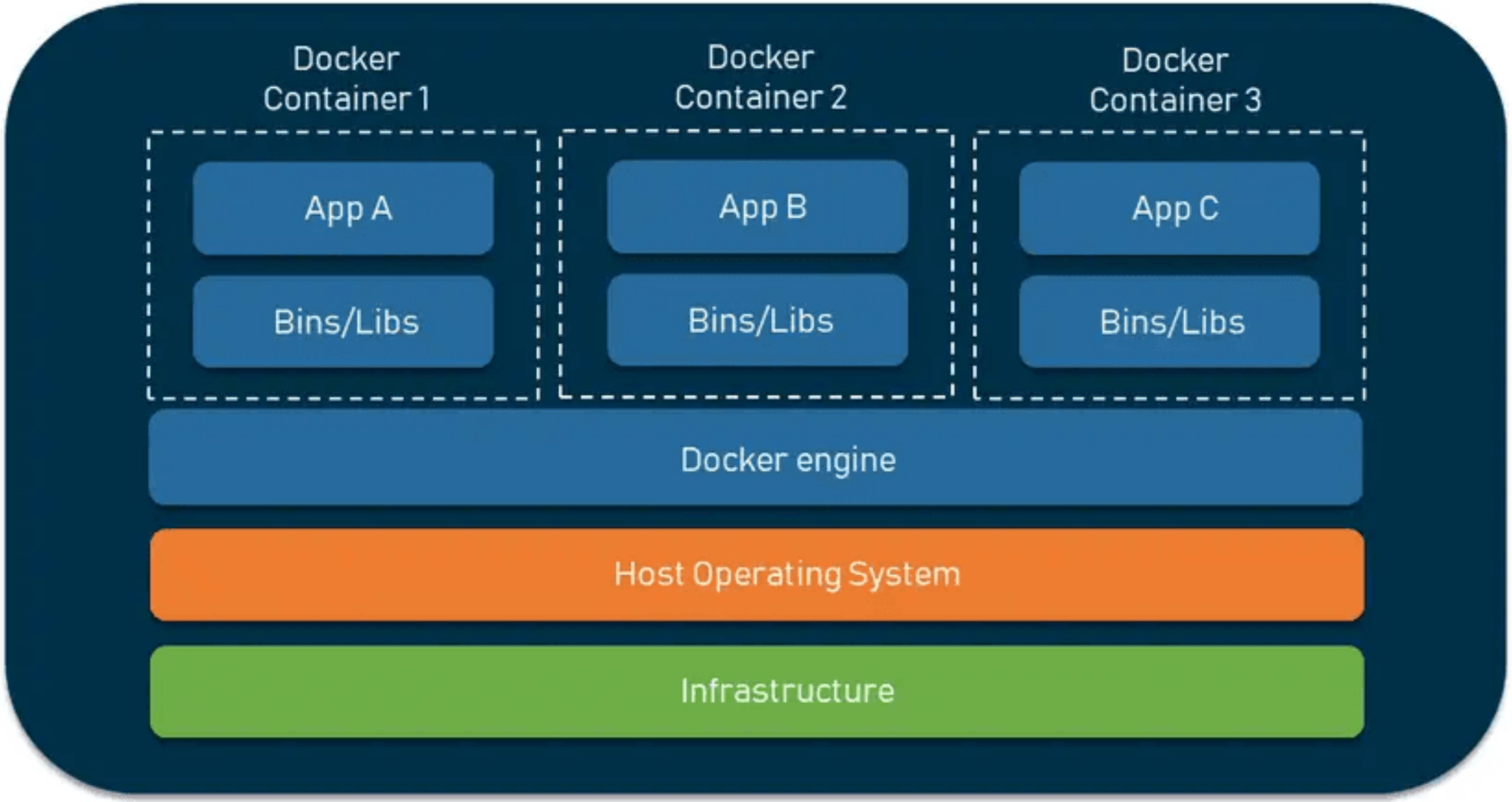
Classification of hypervisors

- Type 1: Bare-Metal Hypervisor (Native)
  - Runs directly on the host hardware, without a host operating system in between.
  - High performance and efficiency; More secure (smaller attack surface); Used mainly in servers and data centers; Manages guest operating systems directly
- Type 2: Hosted Hypervisor (Application)
  - Runs on top of an existing host operating system and behaves like a regular application.
  - Simple setup and configuration; User-friendly interface; Ideal for learning, testing, and development; No need for dedicated hardware; Slower performance due to extra OS layer; Less secure than Type 1

Containers

- Docker, Kubernetes, Podman, Linux Containers (LXC/LXD), OpenShift, etc.
- packages applications and their dependencies into containers.
- is not a hypervisor because it does not virtualize hardware
- shares the host operating system’s kernel.

DOCKER CONTAINERS



Containers vs VMs

Feature	Containers	Hypervisor (VMs)
Virtualization type	OS-level	Hardware-level
Kernel	Shared	Separate per VM
Resource usage	Lightweight	Heavy
Startup time	Seconds	Minutes
Performance	Near-native	Lower
Example	Docker	ESXi, Hyper-V

What is Docker?

Docker is software for automating the deployment and management of applications in container-enabled environments. It allows you to package an application with its entire environment and dependencies into a container that can be deployed to any system, and also provides a container management environment.

Imperfections of traditional software installation technology

- Multiple dependencies on previously installed software
- A separate dependency stack for each software product
- Different versions of a single product required for different software
- Difficulty reproducing the required environment on another computer

Example of COMPSs

- Software installation command line:

  https://compss-doc.readthedocs.io

sudo apt-get install -y openjdk-8-jdk graphviz xdg-utils libtool automake build-essential python2.7 libpython2.7 python3 python3-dev libboost-serialization-dev libboost-iostreams-dev libxml2 libxml2-dev csh gfortran GMP flex bison texinfo python3-pip libpapi-dev

Imperfections of virtual machines

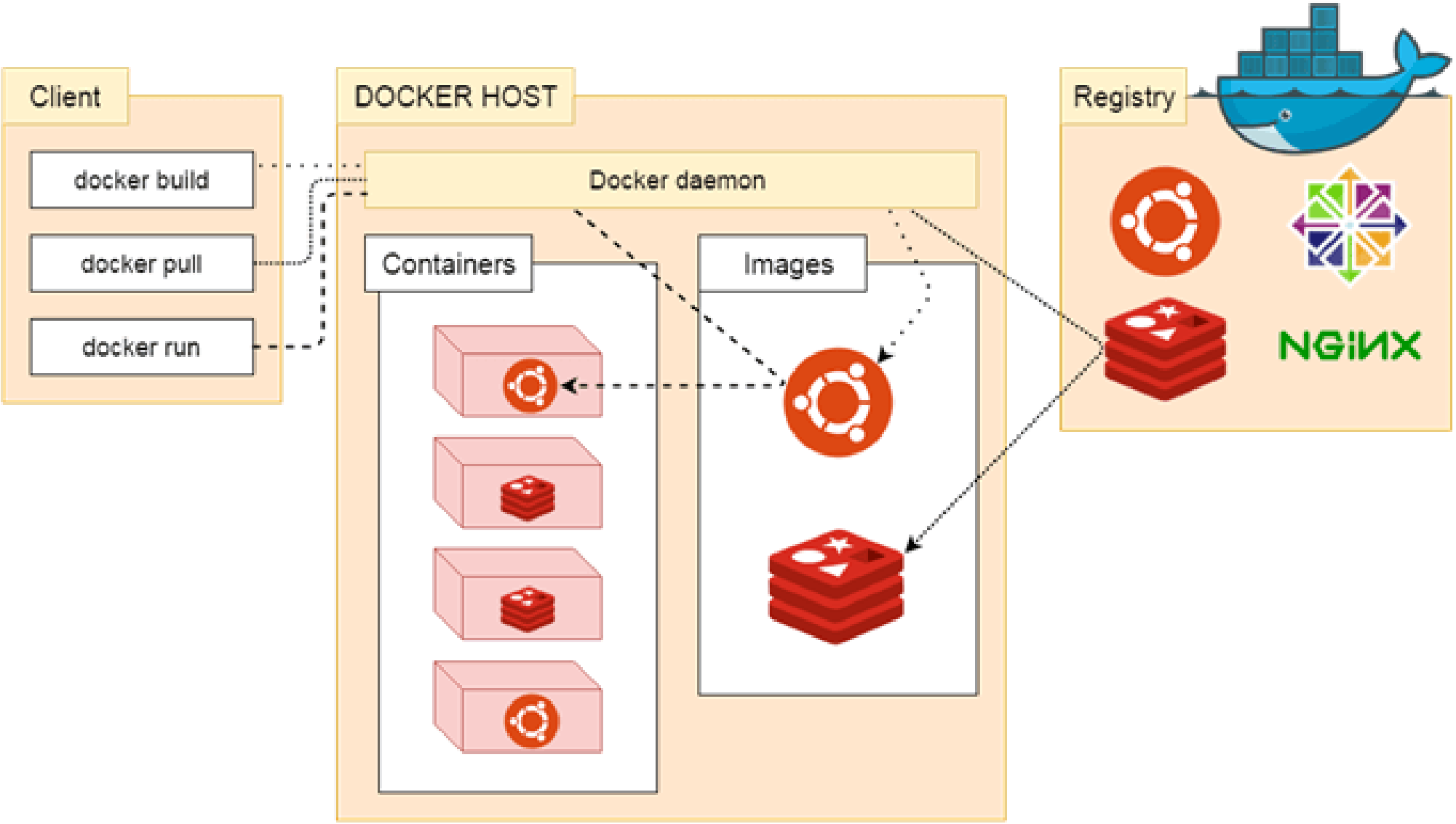
- Slow boot;
- Possible additional storage fees;
- Not all virtual machines (VMs) support compatible use;
- Supported VMs often require complex configuration;
- The image may be too large.

Concept of Docker

- Slim virtual machine
- Image of Docker software
- File Dockerfile – how to assemble image
- Docker container – launch of image in isolated environment

  (docker run image_name)
- Repository of Docker images - Docker Hub (https://hub.docker.com)

Docker Architecture



Install Docker in Ubuntu

- curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add –
- sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable«
- sudo apt-get update
- apt-cache policy docker-ce
- sudo apt-get install -y docker-ce
- sudo systemctl status docker
- docker

- docker run hello-world
- docker search ubuntu
- docker ps -a

Key features of Docker

- Create images, which contain our applications together with all their dependencies;
- Create containers from images to run applications;
- Distribute images via Docker Hub or another image registry.

Example of container launch

- **sudo docker run -i -t ubuntu /bin/bash**

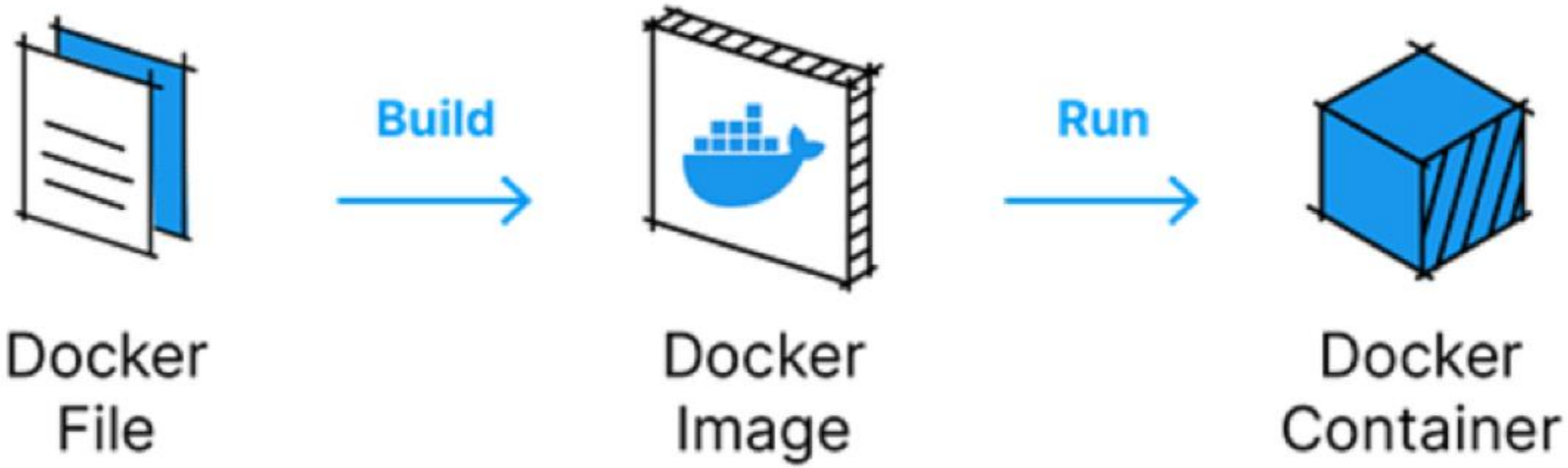
- Downloads the Ubuntu image;
- Creates a container;
- Initializes the filesystem and mounts the read-only layer;
- Initializes the network/bridge: creates a network interface that allows Docker to communicate with the host machine;
- Sets IP addresses: Finds and sets the address;
- Starts the specified process: Launches your application;
- Processes and displays the application's output.

Test container operation

- **docker run ubuntu /bin/echo 'Hello world‘**

```
Unable to find image 'ubuntu:latest' locally latest: Pulling from
library/ubuntu d54efb8db41d: Pull complete
f8b845f45a87: Pull complete
e8db7bf7c39f: Pull complete
9654c40e9079: Pull complete
6d9ef359eaaa: Pull complete
Digest:
sha256:dd7808d8792c9841d0b460122f1acf0a2dd1f56404f8d1e56298048885e4
5535
Status: Downloaded newer image for ubuntu:latest Hello world
```

How to build and run image



Dockerfile structure

- FROM – set the base image
- RUN – run a command in the container
- ENV – set the environment variable
- WORKDIR – set the working directory
- VOLUME – create a mount point for the volume
- CMD – install the executable file for the container

Typical configuration of image, Dockerfile

- FROM <name of existing image>
- COPY <source> <destination>
- RUN <software installation commands>
- CMD <run command>

Dockerfile example

```
FROM ubuntu:latest
RUN apt-get update
RUN apt-get install --no-install-recommends --no-install-suggests -y curl
ENV SITE_URL https://google.com/ WORKDIR /data
VOLUME /data
CMD sh -c "curl -L $SITE_URL > /data/results"
```

Creating an image

- **docker build . -t test-curl**

```
Sending build context to Docker daemon 3.584 kB
Step 1/7 : FROM ubuntu:latest
---> 0ef2e08ed3fa
Step 2/7 : RUN apt-get update
---> Running in 4aa839bb46ec
Get:1 http://archive.ubuntu.com/ubuntu xenial InRelease [247 kB]
Get:2 http://archive.ubuntu.com/ubuntu xenial-updates InRelease [102 kB] ... Fetched 24.9 MB in
4s (5208 kB/s)
Reading package lists...
---> 35ac5017c794
Removing intermediate container 4aa839bb46ec
Step 3/7 : RUN apt-get install --no-install-recommends --no-install-suggests -y curl
---> Running in 3ca9384ecf8d
Reading package lists...
Building dependency tree...
Reading state information...
The following additional packages will be installed...
---> f3c6d26b95e6 Removing intermediate container 3ca9384ecf8d Step 4/7 : ENV SITE_URL
https://google.com/ ---> Running in 21b0022b260f ---> 9a733ee39a46 Removing intermediate
container 21b0022b260f Step 5/7 : WORKDIR /data ---> c024301ddfb8 Removing intermediate container
3bc973e5584c Step 6/7 : VOLUME /data ---> Running in a9594a8958fe ---> 6802707a7114 Removing
intermediate container a9594a8958fe Step 7/7 : CMD sh -c "curl -L $SITE_URL > /data/results" --->
Running in 37503bc4e386 ---> 5ebb2a65d771 Removing intermediate container 37503bc4e386
Successfully built 5ebb2a65d771
```

Run the container and view the results

- docker images
- docker run --rm -v $(pwd)/vol:/data/:rw test-curl
- cat ./vol/results
- docker run --rm -e SITE_URL=https://facebook.com/ -v $(pwd)/vol:/data/:rw test-curl
- cat ./vol/results